1. What AI tools did your team use?
We used Gemini Code CLI, an AI coding assistant that runs in the terminal and can read, write, and run the files in our project.

2. Did the AI tools solve the problem correctly on the first try?
No. Its first version solved the README examples but got stuck on some random dictionaries, so it rewrote the search before we ever submitted. On Gradescope, the first four submissions each failed because the autograder added a new rule: toffee/coconut, the two drizzles, round/heart, and white chocolate groups. Test 1.3 then failed twice more before a version that relaxes possibly replaced rules. The last version passed.

3. What prompt(s) produced the correct solution?
We started with "do lab 10." After that, each prompt was simply the pasted Gradescope output. The AI read which rule had been violated, added it, and retested. The final working version came from pasting the "Need all 25 boxes" failure for test 1.3.

4. Did the results surprise you? Why (not)?
Partly. We expected the AI to write working code quickly, and it did solve the original problem in minutes. What surprised us was how much of the work was testing rather than writing. The AI found and fixed several of its own bugs, such as a retry loop that crashed on small inputs, only because it tested thoroughly. It also couldn't know the hidden rules until the autograder showed them, so feedback still mattered.

5. Do you plan to use AI tools again? Why (not)?
Yes, but maybe a bit more carefully. AI tools are great for writing a first version or trying ideas quickly and running many tests we wouldn't have time to write ourselves. This lab also showed their limits in that the AI couldn't see the grader's dictionary, so it had to guess what the rules meant. In future courses and jobs we'd use AI to speed up routine work,but still check its output ourselves.

6. How can (or can't) AI tools help with debugging?
AI tools help most when they can reproduce the problem. When the code stalled on certain inputs, the AI generated random test cases and found the ones that hung and measured which change fixed them. They are less helpful when the failure can't be reproduced. For test 1.3 it had to figure out from basically no information about a dictionary it couldn't see, so a person (us) still needs to judge whether the fix makes sense.

7. What was the most difficult part of this assignment?
The hardest part was the rules changing after each submission. Every new rule shrank the number of valid boxes, so code that had passed suddenly failed. Test 1.3 was the most difficult, because the autograder only said "Need all 25 boxes" without showing the dictionary. We had to reason about why it might have no answer under our reading of the rules, which was harder than writing the search itself.